

# task01\_project01

---

## Проект «Семейный бюджет»: Требования и План Сбора

### 1. Реестр требований проекта

#### 1.1 Бизнес-требования

*Определяют цели и задачи проекта с точки зрения бизнеса. Почему проект важен и какие проблемы решает..*

1. **Увеличение удержания:** Внедрение сервиса должно повысить показатель удержания клиентов (Retention Rate) на 10% в течение 6 месяцев после запуска за счет повышения «веса» приложения в жизни семьи (эффект сети).
2. **Ограничение по времени и ресурсам:** Релиз MVP должен состояться не позднее чем через 4 месяца от даты старта разработки, с использованием не более 20% текущего ресурса бэкэнд-команды, чтобы не нарушить работу основных банковских систем.

#### 1.2 Пользовательские требования

*Определяют, как продукт должен работать для конечных пользователей, какие функции нужны..*

1. **Прозрачность общих расходов:** Пользователь должен иметь возможность видеть общую картину расходов семьи по категориям (например, «Продукты», «ЖКУ»), не имея при этом доступа к личным транзакциям других членов семьи (сохранение приватности).
2. **Упрощение расчетов:** Пользователь должен иметь возможность быстро зафиксировать, кто и сколько потратил на общую цель, и увидеть автоматически рассчитанную сумму долга для выравнивания бюджета (кто кому сколько должен).

#### 1.3 Функциональные требования

*Подробное описание функций, которые должен выполнять продукт..*

1. **Управление группами:** Система должна позволять пользователю создать «Семейную группу», сгенерировать пригласительную ссылку и добавить участников (максимум 5 человек в MVP), а также исключать участников из группы.
2. **Логика распределения затрат:** Система должна предоставлять функционал ручного добавления транзакции в «Общий котел» с выбором типа распределения (поровну, в процентах или фиксированными суммами) и автоматическим пересчетом балансов участников в реальном времени.

#### 1.4 Технические требования

*Условия реализации: интеграции, производительность, безопасность и др.*

1. **Безопасность данных:** Все данные о семейных группах и транзакциях должны передаваться и храниться в зашифрованном виде (TLS 1.3+, AES-256), доступ к функциям бюджета должен быть

защищен дополнительной биометрической аутентификацией или подтверждением через Push/SMS.

2. **Производительность:** Время отклика интерфейса при обновлении данных бюджета (добавление расхода, пересчет долей) не должно превышать 2 секунд при стандартном мобильном соединении (4G/LTE).

---

## 2. Инструменты и методы сбора требований

Учитывая ограничение в 4 месяца и минимальные ресурсы, нам нельзя тратить время на долгие исследования. Нам нужны методы, которые дают быстрый и качественный результат.

### 2.1 Интервью

- **С кем:** Продуктовый владелец, Руководитель направления безопасности, Представитель маркетинга.
- **Зачем:** Нам нужно четко зафиксировать границы. Безопасники скажут, какие интеграции невозможны за 4 месяца, маркетологи — какие фичи критичны для продвижения.
- **Обоснование:** Это самый быстрый способ получить бизнес-требования и технические ограничения. В условиях сжатого срока личная коммуникация (или видеозвонок) эффективнее долгой переписки, так как позволяет мгновенно прояснять противоречия.

### 2.2 User Story Mapping

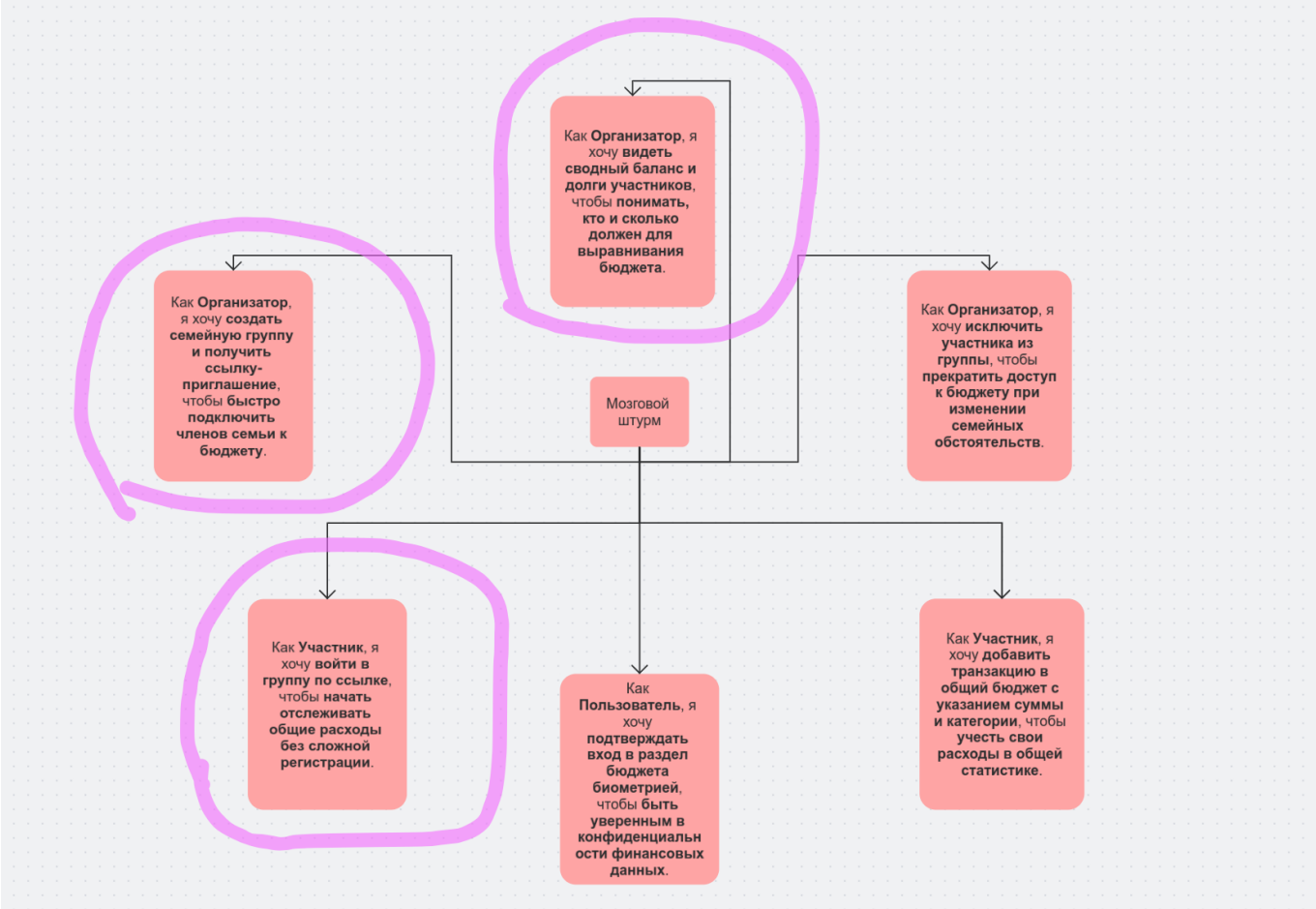
- **С кем:** Команда разработки, Бизнес-аналитик, Дизайнер.
- **Зачем:** Чтобы визуализировать весь путь пользователя и, самое главное, **отсечь лишнее**. Мы разобьем функционал на «Хребет» (обязательно для релиза через 4 месяца) и «Наполнение» (можно отложить на следующие версии).
- **Обоснование:** Этот инструмент критически важен для соблюдения дедлайна. Он помогает команде договориться о том, что именно входит в MVP, избегая «раздувания» функционала, что частая проблема в банковских проектах.

### 2.3 Анализ документов и данных

- **С чем:** Обзоры приложений конкурентов, запросы в службу поддержки банка.
- **Зачем:** Чтобы понять реальные боли клиентов. Возможно, пользователи чаще просят не «бюджет», а «быстрые переводы детям». Это поможет скорректировать пользовательские требования. Также, чтобы не изобретать велосипед, нужно проанализировать, какие похожие продукты уже существуют и возможно взять из них самое лучшее для нашего проекта.
- **Обоснование:** Это метод с минимальными ресурсами. Нам не нужно нанимать дополнительных людей, мы используем уже существующую информацию внутри банка. Это снижает риск сделать продукт, который никому не нужен.

---

## *task02\_project01*



|                       | Доступ                                                                                        | Управление группой                                                                                                 | Учёт расходов                                                                                          | Аналитика                                                                                                    | Минимально жизнеспособный продукт MVP |
|-----------------------|-----------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|---------------------------------------|
| Обязательно сделать   | <div><div>- Вход в приложение</div><div>- Авторизация в приложении</div></div>                | <div><div>- Создание семейной группы</div><div>- Отправка приглашений</div><div>- Принятие приглашений</div></div> | <div><div>- Фиксировать расходы, относящиеся к семье</div><div>- Выбрать категорию расхода</div></div> | <div><div>- Просмотреть свой баланс</div><div>- Видеть долги участников</div></div>                          |                                       |
| Рекомендуется сделать | <div><div>- Настройки уведомлений</div><div>- Биометрический вход</div></div>                 | <div><div>- Исключить участника</div><div>- Назначить организатора</div></div>                                     | <div><div>- Добавить описание к транзакциями</div><div>- Прикрепление чека</div></div>                 | <div><div>- Фильтр по категориям</div><div>- График расходов</div><div>- Экспорт отчёта</div></div>          |                                       |
| Можно сделать         | <div><div>- Обучение пользованием приложением</div><div>- Быстрый вход без пароля</div></div> | <div><div>- Несколько групп</div><div>- Права доступа для участников</div><div>- Гостевой доступ</div></div>       | <div><div>- Голосовой ввод</div><div>- Сканирование чеков</div><div>- Повторяющиеся траты</div></div>  | <div><div>- AI-аналитика</div><div>- Прогноз расходов</div><div>- Сравнение с прошлыми периодами</div></div> |                                       |

Ссылка на таблицу

<https://board.vk.com/?uid=64547b6a-8695-43c6-b443-1d0a8acc50f2>

QRCode на таблицу



---

## *add\_task\_project01*

---

### Стратегия управления проектом «Семейный бюджет»

#### 1. Оценка условий проекта

На основе контекста (сжатые сроки, мобильное приложение, интеграции с банком) выделяю ключевые характеристики, влияющие на выбор методологии:

- **Уровень неопределенности:** Средний. Мы хотим внедрить новую фичу, поведение пользователей неизвестно, нужны быстрые гипотезы и проверка. Значит мы будем возвращаться к некоторым шагам по несколько раз, чтобы доработать неисправности и неудобства, которые могут появиться во время проверок.
- **Стабильность требований:** Низкая. Требования будут уточняться по мере разработки и получения фидбека. Значит у нас не получится составить чёткий план, придётся импровизировать.
- **Вовлеченность заказчика:** Высокая. Бизнес заинтересован, готов участвовать в оценке демо и приоритизации (процесс решения, какие функции делать в первую очередь, а какие отложить). Значит возможна методология Agile с короткими циклами обратной связи.
- **Необходимость быстрой поставки ценности:** Высокая. Через 4 месяца нужен рабочий MVP, а для того, чтобы продукт был актуален и удобен для пользователей, необходимо хотябы раз в две недели предоставлять демо. Значит более удобная методология Agile.
- **Технические ограничения:** Средние. Необходима интеграция с ядром банка, нужно соответствовать требованиям безопасности. Необходима синхронизация с банком, а значит и более структурированный подход.
- **Команда:** Опытная. Знакомы и с Waterfall, и с Agile. Значит можно выбрать оптимальный или гибридный подход.

## Вывод

Нельзя пользоваться только одной методологией. Из-за неопределённости в требованиях необходимо использовать более гибкий способ, такой как Agile, но из-за сжатых сроков в 4 месяца и необходимостью синхронизировать работу с банком, нужно использовать более строгий подход, такой как Waterfall.

Учитывая всё вышесказанное можно сказать, что необходим гибридный подход, который будет совмещать в себе строгость и гибкость.

## 2. Выбор подхода к управлению проектом

Выбранный подход: **Hybrid Agile (Scrum с элементами Waterfall)**

**Scrum** как основной фреймворк для команды разработки + **Waterfall-элементы** для внешних зависимостей (безопасность, интеграции, комплаенс).

### Структура подхода

#### Планирование (Waterfall-элементы):

- Архитектура интеграций (месяц 1)
- Требования безопасности
- Фиксация границ MVP

#### Разработка (Scrum-спринты):

- 2-недельные спринты  $\times 8 = 4$  месяца
- Дейли, демо, ретроспективы
- Гибкая приоритизация бэклога

#### Релиз (Waterfall-элементы):

- Фиксированное окно релиза
- Предварительное тестирование
- Согласование с Compliance

### Почему этот подход?

1. Спринты дают предсказуемый ритм и контроль каждые 2 недели, а срок всего 4 месяца.
2. Итерации позволяют уточнять требования по ходу работы, так как на старте проекта у нас много неопределённостей по поводу итогового продукта.
3. Архитектура фиксируется в начале, чтобы избежать блокировок, так как мы работаем с банком и наше приложение будет интегрировано в его работу.
4. Требования согласуем заранее, реализуем постепенно, поскольку у банка есть определённые требования безопасности.
5. Демо в конце спринта, получаем ранний фидбек и корректировка курса. Нам это нужно, так как вовлечённость бизнеса высокая и нам необходимо показывать, пусть ещё не готовый продукт, но хотя бы демо, чтобы понимать, в ту сторону мы идём или нет.

6. Scrum фокусирует команду на самом ценном, так как у нас ограниченные ресурсы, в том числе и время.

Почему не чистый Waterfall?

- Риск сделать «не то» — дорого, переделывать поздно.
- Нет раннего фидбека от пользователей.
- Изменения ломают весь каскадный план.

Почему не чистый Agile?

- Agile не даёт фиксированных точек для демо заказчику.
- Без предварительного согласования интеграций — риск блокировок.

3. Определи зону MVP

Ниже представленна таблица из второго задания, где был определён MVP (минимально жизнеспособный продукт).

|                       | Доступ                                                           | Управление группой                                                             | Учёт расходов                                                             | Аналитика                                                                | Минимально жизнеспособный продукт MVP |
|-----------------------|------------------------------------------------------------------|--------------------------------------------------------------------------------|---------------------------------------------------------------------------|--------------------------------------------------------------------------|---------------------------------------|
| Обязательно сделать   | - Вход в приложение<br>- Авторизация в приложении                | - Создание семейной группы<br>- Отправка приглашений<br>- Принятие приглашений | - Фиксировать расходы, относящиеся к семье<br>- Выбрать категорию расхода | - Просмотреть свой баланс<br>- Видеть долги участников                   |                                       |
| Рекомендуется сделать | - Настройки уведомлений<br>- Биометрический вход                 | - Исключить участника<br>- Назначить организатора                              | - Добавить описание к транзакциями<br>- Прикрепление чека                 | - Фильтр по категориям<br>- График расходов<br>- Экспорт отчёта          |                                       |
| Можно сделать         | - Обучение пользованием приложением<br>- Быстрый вход без пароля | - Несколько групп<br>- Права доступа для участников<br>- Гостевой доступ       | - Голосовой ввод<br>- Сканирование чеков<br>- Повторяющиеся траты         | - AI-аналитика<br>- Прогноз расходов<br>- Сравнение с прошлыми периодами |                                       |

В первый релиз войдут:

- Вход в приложение, возможность авторизации в приложении. Без возможности войти никто не сможет пользоваться приложением.
- Создание семейной группы, отправка и принятие приглашений. Без группы нет сервиса.
- Фиксировать и выбирать категорию расходов. Учёт и подсчёт денежных средств - основная фишка нашего сервиса.
- Просмотр своего баланса, доступ к долгам участников. Опять же является фишкой нашего сервиса, видеть число доступных средств и "долги", которые имеют участники группы.

4. Краткий план взаимодействия с заказчиком и командой на первых двух неделях проекта

Неделя 1: Инициация

На первой неделе необходимо не просто начать работать над проектом, нужно создать фундамент на котором будет вестись вся последующая работа, поэтому нужно будет провести несколько встреч в онлайн и оффлайн формате для определения и согласования целей.

Встречи:

1. **Цель** Согласовать видение и границы MVP. **Формат** Онлайн, 1.5 часа. **Участники** Команда, владелец продукта, стейкхолдеры.
2. **Цель** Согласовать интеграции и ограничения. **Формат** Офлайн, 2 часа. **Участники** Tech Lead, Backend, Security.
3. **Цель** Разбиение большой задачи на маленькие. **Формат** Онлайн, 1 час. **Участники** Владелец продукта, команда, аналитики.
4. **Цель** Определить цели проекта и приступить к конкретным задачам. **Формат** 1.5 часа офлайн. **Участники** Команда, владелец продукта.

## Неделя 2: Запуск

На второй неделе будет продолжаться процесс синхронизации продукта с требованиями банка. Будут уточняться требования, согласовываться дизайн и под конец недели будет показан первое демо.

### Встречи:

1. **Цель** Синхронизация работников **Формат** Онлайн (если работа на удалёнке), 15 минут ежедневно **Участники** Команда.
2. **Цель** Уточнить требования к авторизации и входу в приложение **Формат** Онлайн, 45 минут **Участники** Security, Backend, владелец продукта.
3. **Цель** Согласовать дизайн интерфейса **Формат** Офлайн, 1 час **Участники** Дизайнер, Frontend, владелец продукта.
4. **Цель** Показать прогресс, получить обратную связь **Формат** Онлайн, 30 минут **Участники** Стейкхолдеры, владелец продукта.

### Как собирается обратная связь?

- Предоставление демо в конце каждой второй недели в формате видео.
- Встреча со спонсором после показа демо.
- Ежедневная синхронизация работников, рассказывают кто что делает и кому что необходимо.
- Получение информации от бета-тестеров.
- In-апп фидбек - сбор обратной связи от пользователей непосредственно внутри мобильного приложения или на сайте.
- Синх-митинги по интеграциям - регулярные рабочие встречи (синхронизации) IT-специалистов, разработчиков, аналитиков или менеджеров, направленные на согласование действий, обмен данными и решение проблем при объединении разных программных продуктов, систем или модулей в единый рабочий комплекс.

### Как фиксируется прогресс?

- Еженедельные статус-отчёты от руководителя проекта.
- Физическая доска спринта, где зафиксированы все выполненные задачи и те, что находятся в процессе.
- YouTrack - мощная система управления проектами и баг-трекер для IT-команд, позволяющая вести Agile-доски (Scrum/Канбан), отслеживать задачи, фиксировать время и автоматизировать процессы.

- Burndown Chart - график в Agile-разработке, показывающий объем оставшейся работы в проекте или спринте по дням. Он наглядно демонстрирует прогресс: горизонтальная ось — время, вертикальная — объем задач (в часах или стори-поинтах). Помогает команде видеть, успевают ли они к дедлайну.
- Confluence и Notion - современные цифровые рабочие пространства (вики-системы) для совместной работы, хранения документации и ведения проектов.
- Release notes - документация, сопровождающая обновление программного обеспечения, в которой описываются новые функции, исправления ошибок, улучшения производительности и изменения в продукте.